

System Detective

CIE-1 — Computer Organisation & Operating Systems (CO1)

Student: Hardik Bhalekar **Environment:** Ubuntu/WSL on Windows

Objective. Inspect the machine with Linux tools, observe processes and system calls, and demonstrate memory locality with a 2D array. The dashboard presents data from the running environment.

1. Machine and ISA

The Iscpu capture reports x86_64, little-endian operation and 4 logical CPUs. The processor is an Intel(R) Core(TM) i3-4010U CPU @ 1.70GHz with 2 cores and 2 threads per core. Microsoft is reported as the hypervisor vendor. Cache sizes shown are 64 KiB L1d, 64 KiB L1i, 512 KiB L2 and 3 MiB L3.

```
(venv) Hardik1214@WINDOWS-T590676: ~ $ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          39 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 4
On-line CPU(s) list:   0-3
Vendor ID:              GenuineIntel
Model name:             Intel(R) Core(TM) i3-4010U CPU @ 1.70GHz
CPU family:             6
Model:                  69
Thread(s) per core:    2
Core(s) per socket:    2
Socket(s):              1
Stepping:               1
BogoMIPS:               3392.14
Flags:                  fpu vme de pse tsc mtr pae mce cmov pat pse
                        ant tsc rcp apic apic xtopology cpuid pni pclmulqdq sse3 fma cx16 pc
                        hf_lm abm invpcid_single pti ssbd ibrs ibpb stibp fsgsbase bmi1 avx2
                        smep bmi2 erms invpcid xsaveopt md_clear flush_l1d arch_capabilities

Virtualization features:
Hypervisor vendor:     Microsoft
Virtualization type:   full
Caches (sum of all):
  L1d:                  64 KiB (2 instances)
  L1i:                  64 KiB (2 instances)
  L2:                   512 KiB (2 instances)
  L3:                   3 MiB (1 instance)
Vulnerabilities:
  Itlb multihit:        KVM: Mitigation: VMX unsupported
  Mlrr:                 Mitigation: PTI
  Mds:                  Mitigation: Clear CPU buffers; SMT Host state unknown
  Meltdown:             Mitigation: PTI
  Spec store bypass:    Mitigation: Speculative Store Bypass disabled via prctl and seccomp
  Spectre v1:           Mitigation: usercopy/swapgs barriers and __user pointer sanitization
  Spectre v2:           Mitigation: Full generic retpoline, IBPB conditional, IBRS_FW, STIBP
                        conditional, SSB filling
  Srbds:               Unknown: Dependent on hypervisor status
  Tsx async abort:      Not affected

(venv) Hardik1214@WINDOWS-T590676: ~ $
```

Figure 1. Actual Iscpu capture.

The ISA and cache hierarchy matter because x86-64 defines the execution model while cache organisation affects memory-access cost. Little-endian byte order stores the least-significant byte at the lowest address.

2. Memory state

At the captured instant, free -h reported 6.1 GiB total, 332 MiB used, 4.4 GiB free, 1.7 GiB buff/cache and 5.6 GiB available. Swap was 2.0 GiB total and 0 B used.

```
(venv) Hardik1214@WINDOWS-T590676: ~ $ free -h
              total        used        free      shared  buff/cache   available
Mem:          6.1Gi          332Mi          4.4Gi          72Ki          1.7Gi          5.8Gi
Swap:         2.0Gi           0B           2.0Gi
(venv) Hardik1214@WINDOWS-T590676: ~ $
```

Figure 2. Actual free -h capture.

3. Processes, threads and states

The top capture shows the WSL environment and its current processes. `ps -eLf` prints thread-level entries: PID identifies the process and LWP identifies a thread. STAT describes scheduling state; S is interruptible sleep and R is running/runnable.

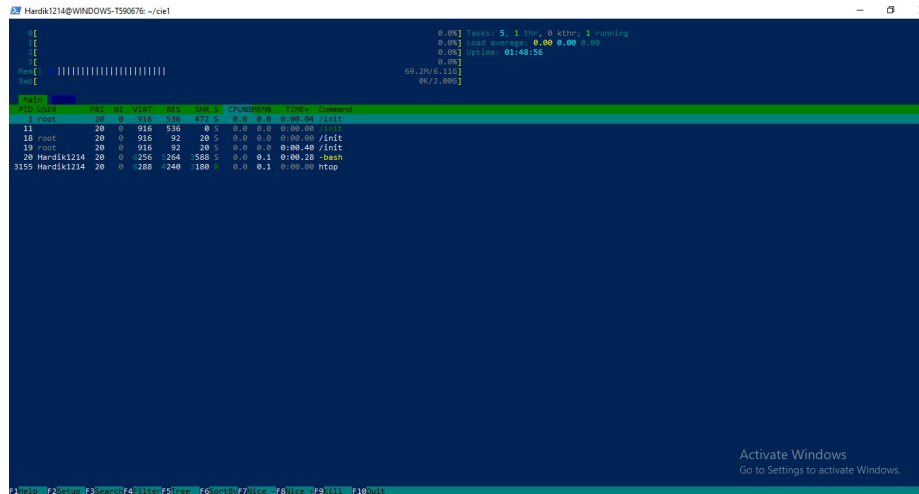


Figure 3. Actual top/htop capture.

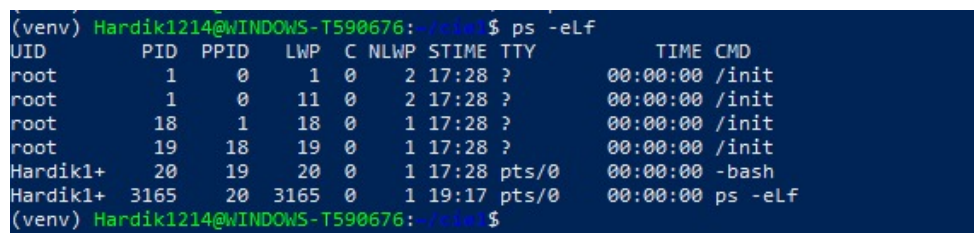


Figure 4. Actual ps -eLf capture.

4. User–kernel boundary with strace

`strace -c ./test` summarizes system calls made during execution. The captured output includes read, write, close, mmap, mprotect, munmap, brk, pread64 and execve. System calls are the interface through which a user-space program requests kernel services.

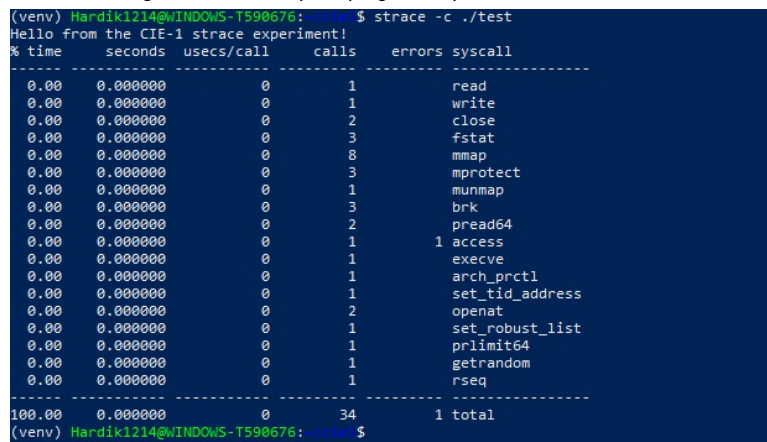


Figure 5. Actual strace -c summary.

5. Locality of reference

The program uses a 4000 × 4000 array. Row-major traversal follows C's storage order, so consecutive accesses are close in memory. Column-major traversal jumps by a full row. Both perform the same high-level work, but their access patterns interact differently with cache lines and prefetching.

Measured results: row-major mean = 0.158465 s; column-major mean = 0.954159 s; column/row ratio = 6.02×.

Run	Row-major (s)	Column-major (s)
1	0.091525	0.509816
2	0.290383	1.848290
3	0.093486	0.504370
Mean	0.158465	0.954159

```
(venv) Hardik1214@WINDOWS-T590676:~$ nano test.c
(venv) Hardik1214@WINDOWS-T590676:~$ gcc test.c -o test
(venv) Hardik1214@WINDOWS-T590676:~$ ./test
Array size: 4000 x 4000
Row-major time: 0.091525 seconds
Column-major time: 0.509816 seconds
Checksum: 6398400000.00
(venv) Hardik1214@WINDOWS-T590676:~$ ./test
Array size: 4000 x 4000
Row-major time: 0.290338 seconds
Column-major time: 1.848229 seconds
Checksum: 6398400000.00
(venv) Hardik1214@WINDOWS-T590676:~$ ./test
Array size: 4000 x 4000
Row-major time: 0.093486 seconds
Column-major time: 0.504370 seconds
Checksum: 6398400000.00
(venv) Hardik1214@WINDOWS-T590676:~$
```

Figure 6. Actual three-run locality experiment.

The second run is much slower than the other two, so the raw runs are retained. The result demonstrates spatial locality: contiguous accesses can reuse nearby data already brought into a cache line, while a large column stride reduces that benefit. The ratio is a measurement of this machine and run, not a universal constant.

6. Dashboard

The Streamlit dashboard combines CPU/ISA information, memory, top processes with state, system-call analysis and the locality test. These captures show it running on localhost:8501.

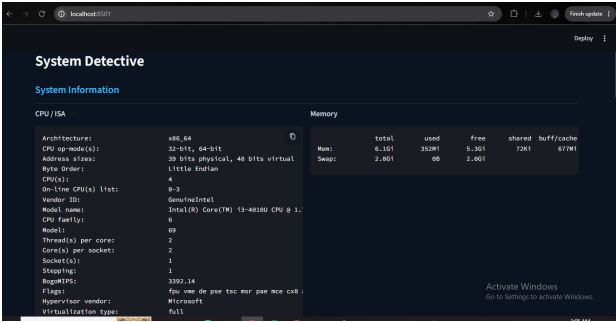


Figure 7. Dashboard — CPU/ISA and memory.

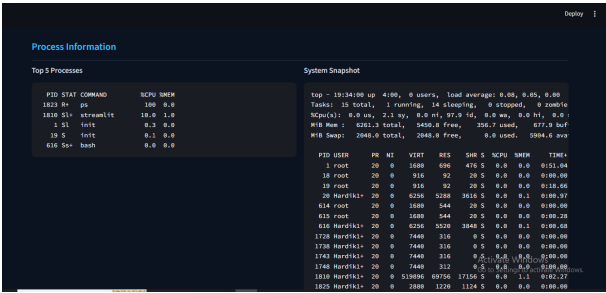


Figure 8. Dashboard — processes and system snapshot.

7. Dashboard evidence and conclusion

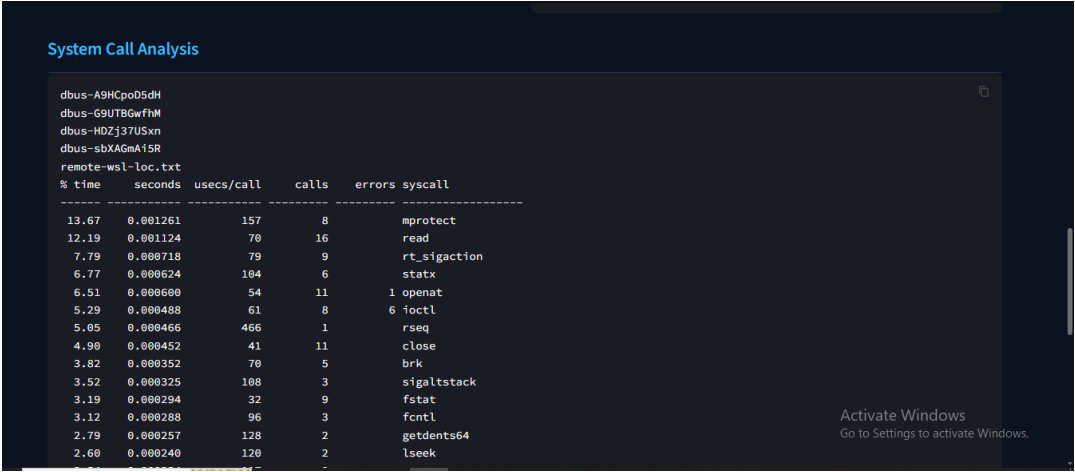


Figure 9. Dashboard — system-call analysis.

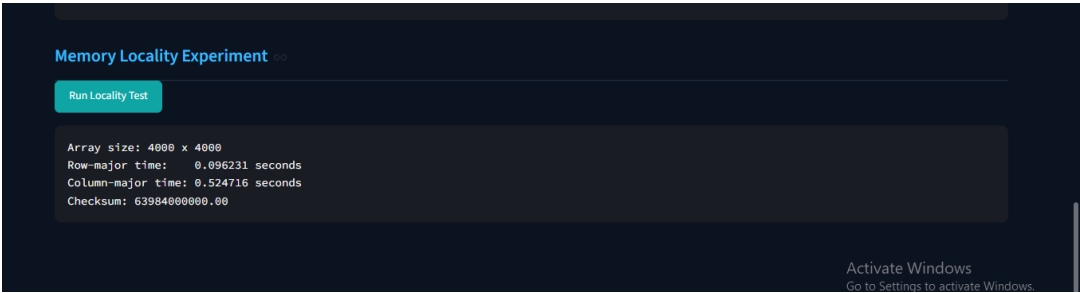


Figure 10. Dashboard — locality test.

Conclusion. The investigation connects computer-organisation concepts with observable OS behaviour. `lscpu` exposes ISA and CPU/cache configuration; `free` exposes memory and swap state; `top` and `ps` expose processes, threads and scheduler states; `strace` exposes system-call activity; and the locality program shows how access order changes runtime despite the same high-level computation.

Reproduction. Re-run the five required commands, compile/run the locality C program, and start the Streamlit dashboard. Process and memory values will change with time, so the raw command outputs and screenshots preserve the observations used here.